Run-Length Encoding (RLE) – Problems with Step-by-Step Solutions

Assumptions (unless otherwise stated): Each original character/pixel is 1 byte. RLE stores each run as (symbol + count). Variant A uses 1 byte for symbol and 1 byte for count. Variant B uses 1 byte for symbol and a 4-bit count (nibble), packing two counts per byte; counts >15 is split.

Q1. Compress 'AAAAABBBCCDAA' and compute sizes/CR.

Runs: A×5, B×3, C×2, D×1, A×2 → total runs = 5. Compressed sequence (logical): A5 B3 C2 D1 A2. Original size = 13 bytes (13 characters).
Compressed size (Variant A) = 5 runs × (1B symbol + 1B count) = 5 × 2 = 10 bytes.
Compression Ratio (CR) = Compressed / Original = 10 / 13 ≈ 0.769 (≈23.1% reduction).

Q2. Binary '1111100000' (length 10). Apply RLE and compute CR.

Runs: 1×5, 0×5 → total runs = 2. Original size = 10 bytes.
Compressed size (Variant A) = 2 × 2 = 4 bytes. CR = 4 / 10 = 0.4 → 60% reduction.

Q3. 'AAAAAAAAAABBBBBBBBBB' (10 A's then 10 B's; N = 20).

Runs: A×10, B×10 → total runs = 2. Original size = 20 bytes.
Compressed size (Variant A) = 2 × 2 = 4 bytes. CR = 4 / 20 = 0.20 → 80% reduction.

Q4. Length 32 with pattern '0000111100001111...' (blocks of 4).

Pattern repeats block of 4 zeros then 4 ones; for N=32 there are 8 blocks → total runs = 8.
Original size = 32 bytes.
Compressed size (Variant A) = 8 × 2 = 16 bytes.

CR = 16 / 32 = 0.5 → 50% reduction (moderate compression).

Q5. Runs = [12, 1, 1, 10, 8, 2, 6] (N = 40). Compare Variant A vs Variant B (nibble counts).

Check sum: 12+1+1+10+8+2+6 = 40.

Variant A: runs = 7 → size = 7 × 2 = 14 bytes → CR_A = 14 / 40 = 0.35. Variant B: all counts ≤ 15 → no splits; runs = 7.
Symbols = 7 bytes; Counts = 7 nibbles = 3.5 bytes → stored as 4 bytes; Total = 7 + 4 = 11 bytes.
CR_B = 11 / 40 = 0.275 → Variant B wins (uses fewer bits for counts).

Q6. N = 50; sequence 'AAAAAAAAAABBBBBCCCCDDDDDDDDD DDDDDDDDDDDDDEEEEFFFFFFFFFF'.

Interpretation by counts: A×10, B×5, C×4, D×17, E×4, F×10 (sums to 10+5+4+17+4+10 = 50).
Runs = 6.
Original size = 50 bytes.

Compressed size (Variant A) = 6 × 2 = 12 bytes. CR = 12 / 50 = 0.24 → 76% reduction.

Q7. Threshold of usefulness for N = 1000. Case A avg run = 3; Case B avg run = 10.

Let K be number of runs; approximately $K \approx N$ / (avg run length).

Case A: $K \approx 1000 / 3 \approx 333.3 \rightarrow$ Compressed $\approx 2K \approx 666.7$ bytes $\rightarrow$ CR $\approx 666.7 / 1000 \approx 0.667$ (saves ~33.3%). Case B: $K \approx 1000 / 10 = 100 \rightarrow$ Compressed = 200 bytes $\rightarrow$ CR = 0.20 (saves 80%).
Conclusion: RLE is increasingly beneficial as average run length exceeds 2; at $\approx$3 it helps modestly, at 10 it helps a lot.

Q8. Nibble storage with overflow. Counts = [3, 7, 9, 20, 15, 1, 14,17].

Splits needed for counts > 15: $20 \rightarrow 15 + 5$; $17 \rightarrow 15 + 2$. Original runs = 8; after splits $\rightarrow$ effective runs = 10.
Variant A (byte/byte): size = 8 × 2 = 16 bytes.

Variant B (nibble counts): Symbols = 10 bytes; Counts = 10 nibbles = 5 bytes; Total = 15 bytes.
Comparison: 15 bytes (B) vs 16 bytes (A) $\rightarrow$ nibble packing slightly better here.

Q9. Alternating binary N = 64: '010101...'. Show expansion and the general rule.

Every run has count = 1 $\rightarrow$ K = 64 runs. Original size = 64 bytes.
Compressed size (Variant A) = 64 × 2 = 128 bytes. CR = 128 / 64 = 2.0 $\rightarrow$ 100% larger (expansion).
General rule (byte/byte RLE): compressed size = 2K; compression occurs iff $2K < N \Leftrightarrow$ average run length = $N/K > 2$.

Q10. Mixed runs N = 60 with Runs = [6, 1, 1, 10, 12, 1, 1, 8, 20]; compare A vs B.
Check sum: 6+1+1+10+12+1+1+8+20 = 60.

Variant A: runs = 9 $\rightarrow$ size = 9 × 2 = 18 bytes $\rightarrow$ CR_A = 18 / 60 = 0.30. Variant B (nibble counts): $20 \rightarrow 15 + 5$ (one split) $\rightarrow$ runs = 10.

Symbols = 10 bytes; Counts = 10 nibbles = 5 bytes → Total = 15 bytes → CR_B = 15 / 60 = 0.25.

Observation: Short runs (1's) hurt compression because each run has fixed overhead; nibble counts mitigate count overhead but not symbol overhead.